

INFUSING ACTIVE LEARNING INTO INTRODUCTORY PROGRAMMING COURSES*

*Keith J. Whittington
B. Gollisano College of Computing and Information Sciences, IT Department
Rochester Institute of Technology
102 Lomb Memorial Drive
Rochester, NY 14623
(585)475-5363
kjlw@it.rit.edu*

ABSTRACT

Since ancient times, it has been known that the best learning takes place when the students are actively involved in the learning process and are critically thinking. As an old Chinese proverb states:

"I hear and I forget. I see and I remember. I do and I understand".

This paper discusses several specific active learning activities that were applied to introductory programming courses. An active and highly interactive classroom was created using these activities. Several of the problems with active learning cited in the literature were also overcome. The effectiveness of these activities was supported by increased student retention, grades, self-confidence, and student surveys.

1. INTRODUCTION

Active learning is based on the constructivist approach to learning where students become active participants in the learning process [2, 5, 6]. This approach is most effective when the students drive the exercise and the instructor guides the process [7].

Although there have been a lot of papers written about active learning, the amount of materials specifically relating to programming is limited. This paper describes various activities used in a programming class that the students felt effectively helped them learn the material and concepts.

* Copyright © 2004 by the Consortium for Computing Sciences in Colleges. Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the CCSC copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Consortium for Computing Sciences in Colleges. To copy otherwise, or to republish, requires a fee and/or specific permission.

Active learning is not supposed to replace the traditional lecture (which still is an effective way to present new material), but to supplement the lecture [7]. It also requires a lot of preparation, courage, and confidence in your mastery of the subject to use this method of teaching [7]. However, the potential rewards include increased student self-confidence, dynamic classroom discussions, and a higher level of student interest in the subject. One of the best ways to gauge whether or not you are succeeding is by the level of noise in the classroom—a quiet classroom is a disengaged classroom.

I found it to be an exhilarating and rewarding experience when an activity has engaged the class. Students start to ask deep, thoughtful questions and test the limits of your knowledge and even cause you to modify your own opinions and knowledge of the subject. Some of the activities I describe in this paper took up the entire class period due to the intense discussions and continual student questions.

That being said, the exercises I describe are not difficult to implement but they do take a lot of pre-planning to make them seem effortless in the classroom. I like to describe this process as planned ad-libbing. There is a lot of flexibility designed into these activities that allow the instructor to flow in the direction determined by the class rather than in a pre-determined path that the traditional lecture provides.

1.1 Background

The exercises discussed in this paper were used in an introductory Java programming sequence designed for the students who struggled in the first programming course. Typically, the second course in the programming sequence presents a significant challenge to a lot of students. So a two-course sequence was offered as an alternative to the traditional second programming course. This allowed these students to take twice as long to cover the same material. The major subjects covered in these courses were encapsulation, polymorphism, program design, collections, I/O, and GUI.

The alternatively paced sequence turned out to be extremely effective and resulted in a dramatic increase in student retention, grades, and self-confidence [13]. Having twice as long to present the materials also gave me the unique opportunity to incorporate several active learning techniques into these courses.

The 42 students who took the alternatively paced sequence the first year it was offered (2002-2003) indicated that the techniques were very effective.

2. COURSE DESIGN

Before I started to cover new material, I wanted to strengthen the students' programming foundation. But to do this I had to determine their current level of understanding. I believe that some students get through the first programming course by memorizing the syntax of the Java language, but this method of learning will not work as well once the concepts become more abstract.

In an attempt to bring the students to a higher level of learning, I started the course by asking why particular constructs were used and what their purpose was, as opposed to asking them how to use various constructs. The first topic discussed was encapsulation,

since it is a fundamental feature of object-oriented programming (OOP) and they were already introduced to it in their first programming course.

If I just repeated the same words they had heard in the first course, I would have had to guess what they didn't understand. Instead, I used various active learning techniques to have the students demonstrate their knowledge. Using these activities as a starting point, we delved deeper into the subject. These activities were so effective that I used them throughout the course. I also developed a rapport with the students as they learned and they seemed to appreciate that they could control (to some extent) the pace of the course.

The following sections describe some of the techniques I used and how I refined them as the courses evolved. The exercises I describe in this paper were the ones I found to be most effective in letting me determine their level of understanding and strengthen their weaknesses. I rated the effectiveness of the various exercises based on student feedback, formative questionnaires, comments, and the general noise level in the classroom.

3. COLLABORATIVE ANSWERS ACTIVITY

It has been well documented that collaboration is a highly effective method to help students learn since it allows them to work together on a problem and helps them reach a mutual understanding [4].

This exercise requires splitting the class into small groups and giving each group a different question. It is very effective at ascertaining the current level of understanding of a topic and provides a great starting point to delve deeper into the material.

One of the problems commonly attributed to active learning is that the students have to come to class well prepared [8]. I alleviate this problem by asking questions about concepts given in previous lectures.

After several iterations of this process, I came up with the following method to administer the questions:

- 1) Randomly split the class into groups of 3.
- 2) Have each group member write an answer to the question as follows:
 - a) One student writes his/her best answer to the question
 - b) The next student provides an answer that improves or adds details to the previous answer
 - c) The third student provides an answer that improves the two previous answers

The students are allowed and encouraged to discuss their answers amongst themselves.

- 3) One group at a time comes in front of the class, and
 - a) Each student presents his/her own answer
 - b) I solicit comments from the class
 - c) I discuss their answers and/or suggest alternative solutions

As previously discussed, I used why-questions versus how-questions in order to elicit higher-level thinking. Typical questions included:

- What is the purpose of constructors?
- Why are attributes defined as private?
- Why are accessors and mutators defined as public?

At first, the groups answered the questions by showing syntax, but they quickly figured out what I was after as I pressed them for better answers.

I wrote a program that split the students into random groups so I could easily change the groups every time I did the exercise. This let them team up with students with various levels of expertise, and form bonds with students that they normally wouldn't have met. The more people students know in a classroom, the more comfortable they are at expressing themselves.

Active learning exercises require many adjustments and refinements as you model them to your particular style and the class dynamics. The following section describes the refinements I made that led to a technique that worked for me.

3.1 Refinements

At first I just handed out the questions to each group, gave them 5 minutes or so to answer them, then had each group answer the question from their desk. One problem with this method was that typically only one student answered the question and the answers were brief and lacked depth. So the next time I had each student provide answers that further qualified the previous answers. This also encouraged the weaker students to volunteer their answer earlier, since it is usually harder to add additional details than to provide an initial answer.

After making this change, I was still disappointed with the results. Even though every student was supposed to supply an answer, typically only one student recited the answers. The answers also tended to be poor, and frequently only two answers were given (I didn't press to find out which student didn't give an answer).

To alleviate this problem, I had each group come up to the front of the class and read their own answers. This got all students participating and gave them an incentive to answer the question as best as they could, since they had to stand in front of the class and read their own answer. This also gave the freshman students some experience at speaking in front of a group of people.

3.2 Observations

Once I refined my technique and the students started to feel comfortable with the process, it became a highly effective tool to let me assess their knowledge. Typically, very few groups got the answers completely correct so I got to explain in detail exactly where they had gone astray. I also carefully chose the questions so each one built from the previous question. When I was particularly inspired I made the questions lead right into the current day's lecture.

I also learned that I couldn't start every lecture with this exercise because it seemed to get stale and I started to see the "oh-this-thing-again" reactions.

4. COLLABORATIVE CODE SOLUTIONS ACTIVITY

An effective variation of the collaborative questions activity is to have the groups write small snippets of code. Every group had a different problem to solve but the steps for this exercise are slightly different:

- 1) Randomly split the class into groups of 3.
- 2) Each group works together on a common solution.
- 3) Every group writes their solution on the board before proceeding.
- 4) One group at a time comes in front of the class, and
 - a) The group explains their solution.
 - b) I solicit comments from the class.
 - c) I discuss their answers and/or suggest alternative solutions.

To save time, I wanted to have all the solutions on the board before I started. This also prevented later groups from changing their answers based on class discussion of the earlier groups.

To make this activity more effective, I numbered each question and arranged them in order of increasing difficulty. This allowed me to progress deeper into the subject as I moved from solution to solution.

4.1 Example

Figure 1 shows the inheritance structure I used for one of these activities, where a Person class is extended by an abstract Employee class that is extended by a SalaryEmployee and a HourlyEmployee class.

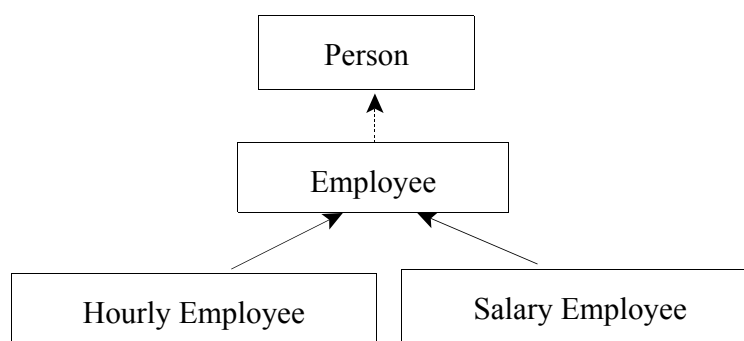


Figure 1: Inheritance Model

This inheritance structure came from their current homework assignment so they were already familiar with it. This allowed them to concentrate on the inheritance problem without first having to understand the basic model.

The objective of the exercise was to deepen the students' understanding of super-classes and sub-classes. To do this I created a series of questions that had them show various ways to call specific methods. For example:

- 1) Given the method signature:

public Employee someMethod(Person p);

Show all possible ways that this method can be called and how you can return the object that is passed to the method.

For this question, a Person, Employee, SalaryEmployee, or Hourly Employee object can be passed to the method, and an Employee, SalaryEmployee, or Hourly Employee object can be returned. In their code they had to check for particular object types, typecast objects, and discuss why you can't create an object from an abstract class.

- 2) Will the following compile?

Employee e = new SalaryEmployee();

If not, show code that will work. Explain your answer.

For this question, they had to discuss abstract objects and class inheritance.

4.2 Observations

Since the majority of the students had errors in their solutions, an interesting development occurred. When I was finished discussing one group's solution, the next group came up to the board armed with an eraser and corrected their mistakes before they discussed their solution.

This was a wonderful way to observe the students actively learning. After I used the above exercise, I noticed that the students were forming their questions better and correctly using phrases such as "is-a" and "has-a".

Sometimes these exercises were so effective and generated so many questions that it took up most of the classroom time. This delayed the planned lecture for the day, but since my main objective was to build strong foundational programming skills, I was willing to pay this price. Plus, since these exercises were used in the alternatively paced courses, I had the luxury of time and I could let the exercises go as long as the students were engaged.

5. PAIRED PROGRAMMING ACTIVITY

Our classrooms are set up so that every student has access to a computer, which allows us to teach our programming classes using a studio model that combines the lecture and lab during the same classroom period. We present a short lecture then give the students an in-class exercise on the materials that were just presented. This model gives us instant feedback for how well the students grasp the newly presented materials.

However, one of the problems with this technique is the inability to field student questions in a timely manner. We typically have 35 students, one instructor, and one TA in each class. Frequently there are 20 or 30 hands raised at the same time, which makes it impossible to answer all the questions. So although the students have the advantage of

an instructor present to help them, they often spend long periods of time waiting for their questions to be answered.

To alleviate this problem, I decided to have the students work together on the exercises. I borrowed the paired-programming technique used as part of the extreme programming model that some companies are currently embracing where two people code together. In this model, one person codes (the driver) while the other person observes (the navigator). However, the navigator doesn't just passively observe the driver, they are an active participant in the process. The navigator analyzes the code, makes design suggestions, and looks for errors. The partners systematically change roles so they both have a chance to be the driver and the navigator [1, 9, 10].

For the same reasons I have previously discussed, I randomly selected new two-person teams for every class. To ensure that each student participated in both roles, I had them change their roles several times throughout the assignment.

5.1 Refinements

Like all the other activities, this activity was continually refined before I was happy with the results. Initially the assignments were divided up into 4 to 5 sections and I had them switch roles at the beginning of each section. Even though each section only took around 10 minutes to complete, I noticed that the person watching quickly lost interest and stopped helping before the section was completed.

To combat this problem, I modified the exercise format so they changed roles more often. I numbered every step of the exercise and told them to switch roles after each number. This simple change seemed to keep everyone attentive. At times I literally had them writing every other line of code.

To promote discussions between the team members, I added questions in the assignments that they had to answer. I also gave them instructions that led them down the wrong path or produced compile errors then included questions that asked them to explain the error. Several students stated that they liked these questions because they knew when they were supposed to have an error and that they had to solve the problem.

5.2 Observations

Even though the students were told to alternate roles, several students were reluctant to be the driver, preferring instead to let the other student do all the typing. For this technique to work every student had to spend time in both roles, so I kept an eye on all the groups and made sure they were all switching roles and taking advantage of their partner's help. It wasn't long before everyone was willingly switching roles.

The most dramatic effect of this alteration of the in-class exercises was the reduction in the number of questions that were asked of the instructor or assistants. The students were now discussing the problems amongst themselves and figuring out most of the problems on their own. Previously, they would have raised their hand and waited for the instructor to answer their questions. They also asked more "what-if" questions, as they explored the new topics.

6. SEQUENTIAL PROGRAMMING ASSIGNMENTS

Another constructivist view of learning is that learners construct new knowledge based on their prior knowledge [3, 6]. I used this theory for their weekly programming assignments by making each assignment build on the previous one. I also started to use the programming assignments' structures in the active learning activities.

This helped generate intelligent questions when new materials were presented because they were always thinking about their current assignment and how the new material would be integrated into their next assignment. The assignments progressed through arrays, collection classes, exception handling, inheritance, abstract classes, interfaces, user-defined exceptions, I/O, and GUI.

By the end of the course, the students had created fairly complex programs consisting of thousands of lines of code. They were also motivated to listen to my additional lessons on modularization, simplification, and elimination of duplicate code, as their code was getting harder and harder to control. Had I discussed these topics prior to their actually experiencing the difficulties of managing more complex code, my advice would have fallen on deaf ears.

6.1 Observations

Some students complained that because each assignment built on the previous ones, if they fell behind on one assignment they wouldn't be able to do the next one. I countered this argument by telling them that since programming concepts build on each other, you're in trouble anyway if you fall behind. Once we were several weeks into the assignments, most of the students agreed that the sequential programming assignments were helping them learn the materials better, and the majority of them kept up with the weekly assignments.

I think that this method of programming assignments was better at keeping students on task and deepened their understanding of OOP concepts. It certainly prompted lots of discussions. I had to spend time in the beginning of most classes going over the various problems they had on the current assignment. However, these questions were usually thoughtful and focused the current topic.

7. SPECIFIC ACTIVITY USED TO TEACH INTERFACES

My previous attempts to get students to understand why interfaces are useful were not successful, so I used an active approach to teach this concept.

As usual, I split the class into groups of three and gave each group an exercise, but this time I used the same premise for each group's exercise. They were told that they were part of a team of programmers assigned to create a graphics program that displayed various animals in an animated fashion.

Each group was given a different animal to define, and were instructed to list various attributes and behaviors for their particular animal. To ensure participation by everyone, each student had to define two unique properties and two unique behaviors for their animal. Every group came to the board and listed their animal, its attributes, and its behaviors.

Once every group had written their solution on the board, I took on the role of the programmer that had to create a program using all the animals listed on the board. First I asked the class to find the common behaviors and attributes and I made a list of these on the board. Next I discussed how I needed to make all the animals appear on the screen and do various things. I randomly chose one of the common behaviors, and discussed how easy it would be if I could create any animal object and call a common method name. This way different groups of programmers could go away and independently create their specific animal, and when they were finished I could easily integrate their animal into the program because I already knew the names of the behaviors and attributes. For instance, if I wanted three different animals to run across the screen, I all I had to do was call `dog.run()`, `cat.run()`, and `bird.run()`.

I discussed the power of being able to enforce specific types of behaviors and attributes on each animal without being concerned with the details of each implementation. At this point, I created a class called `Animal` on the board with all the common attributes and behaviors. Then I added the required syntax to make this an interface and discussed how Java forces you to define all of the methods and attributes when you create a class that implements the interface.

7.1 Observations

The students really seemed to relate to this exercise and actually guided me along the path to defining the interface. I heard a lot of ah-ha's during the discussion, and they even got a little insight into program design without me specifically mentioning the word "design".

8. RESULTS

In addition to a subjective analysis of the student's feelings and attitudes, a questionnaire was given that asked the students to rate how effective each activity was in helping them understand programming. Forty-two students in the two sections offered of these new courses participated in the survey. Table 1 shows the results of the questionnaire.

Table 1: Student Questionnaire Results (42 students)

Activity	Effective	Not Effective/ Unknown
Group questions / discussion	93%	7%
Asking "why" questions	90%	10%
Code examples on the board	71%	29%
Paired Programming	90%	10%

The active learning exercises were well received. One of the surprising results was that 93% of the students rated the group questions as an effective technique that helped them learn, even though they had to get in front of the class and discuss their answers.

Fewer students liked the exercises where they had to show their answers on the board, but with an approval rate of 71% I believe it was still effective. Plus, several students wrote in their review of the class that they felt that these exercises were their favorites. The survey also indicated that 90% of the students felt that "why" questions helped them understand the material better. They also overwhelmingly (90%) chose the paired programming style over the way they had previously done their in-class activities.

9. CONCLUSIONS

One of the motivations for infusing the active learning techniques into these programming courses was that the majority of our students are active learners (based on the Felder-Soloman test) [11]. Studies have shown that active learners typically do worse than reflective learners in introductory programming courses [12].

I found the activities I described to be highly effective. They also created a dynamic environment in which even the shy and/or hesitant students participated in discussions. Coming up with effective exercises is not a trivial task but is well worth the effort since it pushes the students to think and solve complex problems. I found these exercises to be especially effective when used to cover concepts in greater depth after the students have been using them for a while.

I believe that one of the keys to the success of these courses was the programming assignment model where each assignment built on the previous one. This kept a common thread going throughout the course that the students understood and could relate to. This directly addresses the traditional programming exercises problem that Carbone, et al. discuss as a major problem wherein students tend to think of each assignment as a separate entity and don't see the relationship between assignments [3].

Having an assignment that lasted all quarter enabled me to use the data structures in the active exercises, and let them build upon their previous knowledge. The students got to create programs sufficiently large enough to help them see the benefits of the OOP paradigm. I could also probe deeper into OOP concepts since they didn't have to think about a new data structure every time a new exercise was introduced.

My feelings about the programming assignments were confirmed in many of the student's comments:

- I incorporated older material I learned into new homeworks (to strengthen my knowledge and make it all that much more familiar)
- Whenever I ran into problems in new homeworks, I had already encountered the error before and knew EXACTLY how to fix it.
- One of the most effective learning aides was in using/modifying one program for all the assignments. That really helped re-enforce the concepts.

Although these activities take a lot of preparation, it is well worth the effort since it has been shown that active learning enhances student learning. I also understand that most instructors do not have the time to incorporate all of the activities described in this paper, but they can certainly try one of these activities and see how it goes. Slowly incorporating active learning into the classroom is probably the best way to get started

since each instructor will have to modify their activity to match their particular teaching style and the unique dynamics of a particular class.

10. REFERENCES

- [1] Astels, D., G. Miller, and M. Novak, A Practical Guide to Extreme Programming. Prentice Hall. 2002.
- [2] Ben-Ari, M., Constructivism in Computer Science Education. SIGCSE Bulletin, 1998: p. 257-261.
- [3] Carbone, A., et al. Characteristics of Programming Exercises that Lead to Poor Learning Tendencies: Part II. in ITiCSE. 2001. Canterbury, UK.
- [4] Chase, J.D. and E. Okie, Combining Cooperative and Pair Instruction in Introductory Computer Science. SIGCSE Bulletin, 2000: p. 52-55.
- [5] Grissom, S. and M.J. Van Gorp, A Practical Approach to Integrating Active and Collaborative Learning into the Introductory Computer Science Curriculum. Consortium for Computing in Small Colleges, 2000: p. 97-102.
- [6] Hadjerrouit, S. A Constructivist Approach to Object-Oriented Design and Programming. ITiCSE. 1999. Cracow, Poland: ACM Press.
- [7] Huffman, K. and M. Vernoy, Psychology in Action. 7th ed. Wiley 2003
- [8] Lee, J.A.N., Teaching and Learning in the 21st Century: The development of "future CS faculty". SIGCSE Bulletin, 2001. 33(2): p. 25 - 30.
- [9] McDowell, C., et al., The Effects of Pair-Programming on Performance in an Introductory Programming Course. SIGCSE Bulletin, 2002: p. 38-42.
- [10] Nagappan, N., et al., Improving the CS1 Experience with Pair Programming. SIGCSE Bulletin, 2003: p. 359-362.
- [11] Soloman, B. and Felder, R., Index of Learning Styles, <http://www.ncsu.edu/felder-public/ILSpage.html>
- [12] Thomas, L., et al. Learning Styles and Performance in the Introductory Programming Sequence. SIGCSE Bulletin, 2002. p. 33-37.
- [13] Whittington, K., et al. Implementation of Alternative Pacing in An Introductory Programming Sequence. SIGITE 2003. West Lafayette, Indiana.